typesense

Tips for Filtering Data in Typesense

Copy Markdown

In this article, we'll talk about how to filter data in Typesense using various filtering options:

- Basic Filtering
- Available Operators
  - Non-exact Operator for String Types
  - Array Operators
- Filtering for Empty Fields
- Prefix Filtering
- Boolean Operations
  - Operator Precedence
- Filtering Array Types
- Escaping special characters
- Filtering Geopoints
- Filtering Nested Object Fields
- Filtering Nested Array Objects
- Filtering Joined Collections

Basic Filtering

Typesense allows you to filter your search results based on the values of specific fields. You can use the `filter_by` parameter to specify a custom filter string.

```json
{
  "q": "*",
  "filter_by": "country:=USA"
  // ...
}
```

Note that the base format for a filter is `field: <operator> <value>`. Every filter field must have a `:` after it.

Available Operators

Operator	Description	Available types	Example
`=`	Equal to	`string` , `int32` , `int64` , `float` , `bool`	`country:=USA`
`:`	Partially equal to	`string`	`country:New`
`>`	Greater than	`int32` , `int64` , `float`	`price:>100`
`<`	Less than	`int32` , `int64` , `float`	`price:<100`
`!=`	Not equal to	`string` , `int32` , `int64` , `float` , `bool`	`status:!=inactive`
`<=`	Less than or equal to	`int32` , `int64` , `float`	`price:<=100`
`>=`	Greater than or equal to	`int32` , `int64` , `float`	`price:>=100`
`[]`	Is one of	`string` , `int32` , `int64` , `float` , `bool`	`country:[USA, UK, Canada]`
`![]`	Is not any of	`string` , `int32` , `int64` , `float` , `bool`	`country:![USA, UK, Canada]`
`[..]`	Range	`int32` , `int64` , `float`	`price:[100..200]`

Non-exact Operator for String Types

The non-exact operator (`:`) allows for **word-level** partial matching on `string` fields. It's useful when you want to find results that contain a specific word:

```
location:New
```

This filter will match any country name containing "New", such as:

- ✅ New Zealand
- ✅ New Caledonia

- ❌ `Newfoundland`

Since `New` is not a standalone word in the string `Newfoundland`. If you need this type of matching, see Prefix Filtering below.

> **Performance Tip**
>
> The non-exact operator (`:`) doesn't take token position into account, so it is usually faster than the exact `:=` operator.

Array Operators

Array operators allow you to filter numeric fields (`int32` , `int64` , or `float`) based on specific conditions:

```
price:[<20, >100]
```

Will result in items with a price of:

- Under 20 `OR`
- Over 100

Commas act as OR operators, allowing flexible condition combinations.

The Range Operator

The range operator can be used with multiple array elements, creating a logical OR between ranges or values:

```
price:[100..200, 15..50, 800]
```

This matches items with prices:

- Between 100 and 200
- Between 15 and 50

Filtering for Empty Fields

You may want to filter for records where a specific field is empty or null.

Since Typesense doesn't allow an entirely empty filter value in `filter_by`, you can use placeholders instead.

One approach is to use an alphanumeric placeholder (like `~~`) when you want to find empty fields:

```json
{
  "q": "*",
  "filter_by": "optional_field:=~~"
}
```

To do this though, when indexing your data, you would need to:

1. Add your placeholder characters to the `symbols_to_index` parameter in the collection schema.
2. When adding documents, identify fields that might be empty and replace the empty or null values with your chosen placeholder (e.g., `~~`)

```js
// Example preprocessing before indexing
const document = {
  title: "Product name",
  description: "~~", // Empty field
  category: "Electronics"
};
```

Then, to filter for records with empty descriptions:

```
filter_by = description:=~~
```

Similarly, to filter for records with non-empty descriptions, you can use the `!` operator:

```
filter_by = description:!~~
```

Symbols are not indexed by default in Typesense.

So to make the above technique work, you would need to configure the `symbols_to_index` parameter for the specific field in your collection schema.

Here's how to set up your schema with `symbols_to_index`:

```json
{
  "name": "products",
  "fields": [
    {"name": "title", "type": "string"},
    {"name": "description", "type": "string", "symbols_to_index": ["~"]},
    {"name": "category", "type": "string"}
  ]
}
```

By including the desired symbol in `symbols_to_index`, Typesense will properly index and allow filtering on that symbol. You can then use it as a placeholder for empty fields:

Prefix Filtering

> This feature is only available as of Typesense Server v27.0

You can use an asterisk (`*`) to denote a prefix in a `string` field filter. This allows you to filter for fields that **begin with** the provided prefix:

```
name:Jo*
```

This filter will return results for people whose names include a word starting with `Jo`, such as:

- Jonathan Jacobs
- John McKinley
- Michael Johansson

When you use the prefix in conjunction with the exact-match operator `:=`, the results will only include entries where the prefix matches the beginning of the name:

This will match:

- ✅ Jonathan Jacobs
- ✅ John McKinley

But not:

- ❌ Michael Johansson

Boolean Operations

You can combine multiple filters using logical operators.

Operator	Description	Example
`&&`	Logical AND	`country:=USA && city:=New York`
`||`	Logical OR	`country:=USA || country:=Canada`

When using logical operators, you can use parentheses to group your filters.

For example, to filter for products that are either made in the USA or Canada, and are located in New York, you can use the following filter:

```
(country:=USA || country:=Canada) && city:=New York
```

This will return products that are either made in the USA or Canada, and are located in New York. When using logical operators, you can use parentheses to group your filters. For example, to filter for products that are either made in the USA or Canada, and are located in New York, you can use the following filter:

```
(country:=USA || country:=Canada) && city:=New York
```

Operator Precedence

to specify a different order.

For example:

```
country:=USA || country:=Canada && city:=New York
```

Would be interpreted as:

```
country:=USA || (country:=Canada && city:=New York)
```

To change the order of operations, use parentheses:

```
(country:=USA || country:=Canada) && city:=New York
```

Filtering Array Types

Any of the array types can be used for filtering, matching any of their values.

For example, if there's a field called `department_prices` that's a `int32[]` and there's a document with `department_prices: [32, 60, 80]`, then the filter of:

```
department_prices:<80
```

Would in turn return this document.

Subsequently, you can also use array operators:

```
deparment_prices:[20..80]
```

Escaping special characters

If you want to filter for a field *value* that includes special characters, such as a `&&`, `||`, `(`, `)`, `,`, `[`, `]`, you can use backticks around your filter value:

Typesense does not allow special characters in field *names*, only field *values* can be escaped using backticks.

Filtering Geopoints

You can filter on `geopoint` fields by searching either within a specific radius, or by specifying a bounding box.

For example, to filter for results within a 10 kilometer radius of a specific location, you can use the following filter:

```
location:(48.90615915923891, 2.3435897727061175, 10 km),
```

> **TIP**
>
> You can use the same filter for miles by changing the distance unit to `mi` .

Or, to filter for results within a bounding box, you can use the following filter:

```
location:(48.8662, 2.3255, 48.8581, 2.3209, 48.8561, 2.3448, 48.8641, 2.3469)
```

For information on filtering `geopoint` fields to search within a specific area, please refer to our documentation on Geosearch.

Filtering Nested Object Fields

You can also filter on nested object fields by using the dot (`.`) notation to refer to nested fields.

For example, if you have a `customer` nested object inside your collection schema that looks like this:

json

```
{
  "customer": {
```

```
  }
}
```

You can filter by the customer's name like:

```
customer.name:=John Doe
```

Filtering Nested Array Objects

When filtering on fields inside nested array objects, you need to use a special syntax to ensure the filters are applied to the same object within the array. The syntax is: `<nested_field_parent>.{<filter_conditions>}`.

For example, consider a menu collection with the following schema:

```json
{
  "name": "menu",
  "fields": [
    {"name": "name", "type": "string"},
    {"name": "ingredients", "type": "object[]"},
    {"name": "ingredients.name", "type": "string[]"},
    {"name": "ingredients.concentration", "type": "int32[]"}
  ],
  "enable_nested_fields": true
}
```

And these sample records:

```json
{
  "name": "Pasta",
  "ingredients": [
    {"name": "cheese", "concentration": 40},
    {"name": "spinach", "concentration": 10},
    {"name": "jalepeno", "concentration": 20}
  ]
}
{
```

```
      {"name": "pizza sauce", "concentration": 30},
      {"name": "olives", "concentration": 30}
    ]
  }
  {
    "name": "Lasagna",
    "ingredients": [
      {"name": "cheese", "concentration": 60},
      {"name": "jalepeno", "concentration": 20},
      {"name": "olives", "concentration": 20}
    ]
  }
```

To filter for dishes that have cheese with concentration less than 50, use:

```
filter_by=ingredients.{name:=cheese && concentration:<50}
```

This will return only Pasta and Pizza, because it matches objects within the ingredients array where both conditions are true for the same object.

> **Important Note**
>
> The traditional filter syntax `ingredients.name:=cheese && ingredients.concentration:<50` would match all three records because it doesn't ensure the conditions apply to the same object in the array. Always use the `{...}` syntax when filtering on multiple fields within nested array objects.

> **Important Note**
>
> Fields inside `object[]` fields automatically become arrays. So we have to mark `ingredients.name` and `ingredients.concentration` as array fields, despite being single valued.

You can use any valid filter operators inside the `{...}` block.

For example:

```
# Find dishes with cheese concentration between 30 and 50
filter_by=ingredients.{name:=cheese && concentration:[30..50]}
```

Filtering Joined Collections

When filtering for joined collection, you use the `$CollectionName(<filter>)` syntax.

For example, if you have a `products` collection with a `category` field and a `categories` collection with a `name` field, you can filter for products that are in the `Electronics` category like this:

```
$categories(name:=Electronics)
```

Inside of the joined collection, you can use any filter string that you would use directly for that collection, including `&&`, `||` and `!` boolean operators. For example, to filter on products in the Electronics category AND the `Mobile` category, but NOT in the `Disposable` category, and under $100, you can use the following filter:

```
$categories(name:=Electronics && name:=Mobile && name:!=Disposable) && price:<100
```

Filtering on joined fields is supported at any level of nesting. For a detailed explanation and examples, please refer to our documentation on nested joins.

> **WARNING**
>
> Joins must be configured in the schema prior to using them in a filter.

See JOINs for more details.

Edit page

Last Updated: 11/12/2025, 10:34:54 AM

typesense

⌘ K